

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a,b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a,b]$.

Input

The first input line has two integers n and q : the size of the array and the number of queries.

The next line has n values $t_1, t_2, \dots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either " $1 \ a \ b$ " or " $2 \ a \ b$ ".

Output

Print the answer to each sum query.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq t_i \leq 10^6$
- $1 \leq a \leq b \leq n$

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3

EXPECTED OUTPUT

9

Test Case 2

INPUT (STDIN)

5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5

EXPECTED OUTPUT

17
32

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void build(int k,int
l,int r)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

800

Test Case 3

NLOC

71

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define MAXN 200005
6
7 long long tree_sum[4 * MAXN];
8 long long lazy_a[4 * MAXN];
9 long long lazy_d[4 * MAXN];
10 long long initial_arr[MAXN];
11
12 void build(int k, int l, int r) {
13     lazy_a[k] = 0;
14     lazy_d[k] = 0;
15     if (l == r) {
16         tree_sum[k] = initial_arr[l];
17         return;
18     }
19     int mid = (l + r) / 2;
20     build(2 * k, l, mid);
21     build(2 * k + 1, mid + 1, r);
22     tree_sum[k] = tree_sum[2 * k] + tree_sum[2 * k + 1];
23 }
24
25 void apply_lazy(int k, int l, int r, long long a, long long d) {
```

```

26     long long len = r - l + 1;
27     tree_sum[k] += len * a + d * len * (len - 1) / 2;
28     lazy_a[k] += a;
29     lazy_d[k] += d;
30 }
31
32 void push(int k, int l, int r) {
33     if (lazy_a[k] != 0 || lazy_d[k] != 0) {
34         int mid = (l + r) / 2;
35         apply_lazy(2 * k, l, mid, lazy_a[k], lazy_d[k]);
36         apply_lazy(2 * k + 1, mid + 1, r, lazy_a[k] + (mid + 1 - l) * lazy_d[k], lazy_d[k]);
37         lazy_a[k] = 0;
38         lazy_d[k] = 0;
39     }
40 }
41

```

```
42 void update(int k, int l, int r, int ql, int qr) {
```

```
43     if (ql <= l && r <= qr) {
```

```
45         long long d = 1;
```

```
46         apply_lazy(k, l, r, a, d);
```

```
47         return;
```

```
48     }
```

```
49     push(k, l, r);
```

```
50     int mid = (l + r) / 2;
```

```
51     if (ql <= mid) update(2 * k, l, mid, ql, qr);
```

```
52     if (qr > mid) update(2 * k + 1, mid + 1, r, ql, qr);
```

```
53     tree_sum[k] = tree_sum[2 * k] + tree_sum[2 * k + 1];
```

```
54 }
```

```
55
```

```
56 long long query(int k, int l, int r, int ql, int qr) {
```

```

57     if (ql <= l && r <= qr) {
58         return tree_sum[k];
59     }
60     push(k, l, r);
61     int mid = (l + r) / 2;
62     long long res = 0;
63     if (ql <= mid) res += query(2 * k, l, mid, ql, qr);
64     if (qr > mid) res += query(2 * k + 1, mid + 1, r, ql, qr);
65     return res;
66 }
67
68 int main() {
69     int n, q;
70     if (scanf("%d %d", &n, &q) != 2) return 0;
71     for (int i = 1; i <= n; i++) {

```

```
72     if (scanf("%lld", &initial_arr[i]) != 1) initial_arr[i] = 0;
73 }
74 build(1, 1, n);
75 while (q--) {
76     int type, a, b;
77     if (scanf("%d %d %d", &type, &a, &b) != 3) break;
78     if (type == 1) {
79         update(1, 1, n, a, b);
80     } else {
81         printf("%lld\n", query(1, 1, n, a, b));
82     }
83 }
84 return 0;
85}
86
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3

3 7 2 2 5

3 2 3

1 3 6

1 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree26.exe

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

4 2 3 1 7

2 1 5

1 1 5

2 1 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree26.exe

17

32

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>